

Session:2018-2019

1. What is a Von Neumann machine? Write down the major difference between Intel 8085 and 8086 Microprocessor.

A machine that stores the instructions of a program in the memory system. a conceptual model of nearly all computer architecture.

Feature	Intel 8085	Intel 8086
Architecture	8-bit	16-bit
Data Bus	8-bit	16-bit
Address Bus	16-bit	20-bit (later versions supported 16-bit mode)
Registers	74 instructions, 74 opcodes	246 instructions, 191 opcodes
Instruction Set	Limited set of instructions	Rich set of instructions for complex tasks
Registers	Accumulator, BC, DE, HL, SP, PC	General Purpose Registers (AX, BX, CX, DX, SI, DI, BP, SP), IP, Flags

2. What is the difference between an intersegment and intra-segment jump? Show which JMP instruction assembles (short, near or far) if the JMP THERE instruction is stored at memory address 10000H and the address of THERE is:

- I. 10020H
- II. 11000H

Aspect	Intersegment Jump	Intra-segment Jump
Definition	Jump between different code segments.	Jump within the same code segment.
Addressing	Involves a new segment address.	Uses addresses within the current segment.
Memory Access	Can access different memory segments.	Limited to the memory allocated within the current segment.
Usage	Used for accessing code/data in different memory areas.	Typically used for loops, conditionals within the same code.
Example	JMP 2000:0100 (Jumps to segment 2000, offset 0100).	JMP 0100 (Jumps to offset 0100 within the current segment).

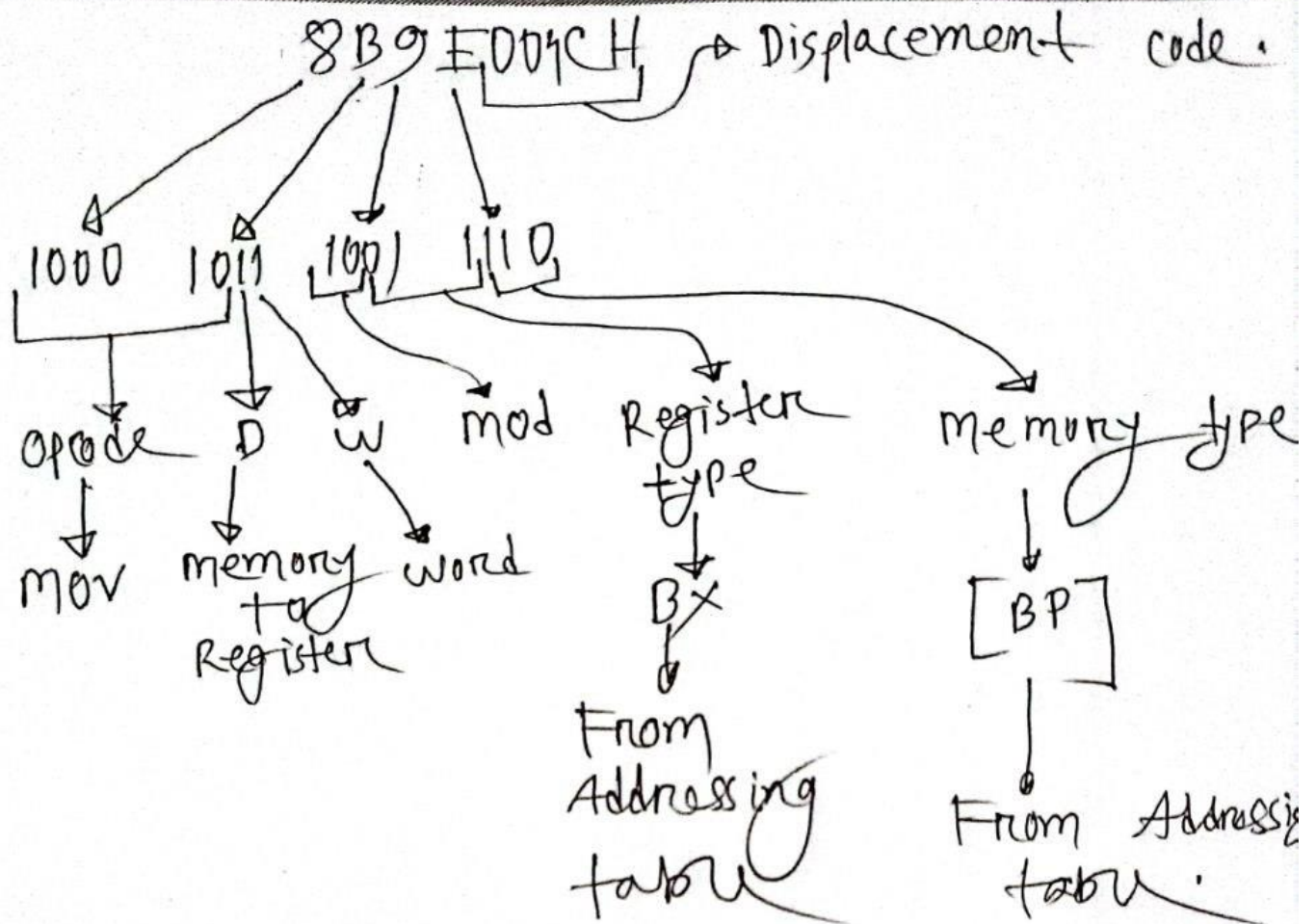
- If the memory address of THERE is 10020H,

10020H
-10000H
00020H → 0000 0000 0000 0010 0000
1 byte
1 byte displacement → short jump

- If the memory address of THERE is 11000H,

11000H
-10000H
01000H → 0000 0001 0000 0000 0000
2 byte
2 byte displacement → near jump

3. Convert an 8B9E004CH from machine language to assembly language. IF a MOV SL[BX+2] instruction appears in a program, what is its machine language equivalent?



So, The Assembly language instruction is:

MOV BX, [BP + Reverse displacement byte]

MOV BX, [BP + 4C00H]

Shammy

- The machine language equivalent for MOV SL,[BX+2] in x86 assembly is: 8B 47 02

4. What will be the CS:IP of physical address BCDEFh where CS=FFF0? How is the local descriptor table addressed in the memory system?

To calculate the CS:IP from a physical address in x86 architecture, we can use the following formula:

Physical address=(CS x 10) + IP

Given that CS is FFF0₁₆ and the physical address is BCDEF₁₆.

Let's calculate the IP:

BCDEF₁₆= FFF0₁₆ x 10+IP

IP= BCDEF₁₆ - FFF0₁₆

IP= -43111

So, the CS:IP pair for the given physical address BCDEF₁₆ where CS is FFF0₁₆ would be FFF0₁₆ : -43111₁₆ in x86 assembly language notation.

The Local Descriptor Table (LDT) is addressed by using a segment selector to index into the Global Descriptor Table (GDT). The segment selector's Descriptor Table Indicator (DTI) bit indicates whether the GDT or LDT is being accessed.

Example:

; Assuming ES holds the segment selector for the LDT

MOV EAX, [ES:0x10] ; Access data at offset 0x10 in the LDT

In this example, ES holds the segment selector for the LDT, and 0x10 is the offset within the LDT where data is being accessed.

5. Which register locates the global descriptor table? Describe the content of the segment register at protected memory addressing.

In the x86 architecture, the Global Descriptor Table (GDT) is a table that defines segment descriptors used for memory access. The GDT is typically loaded into the GDTR (Global Descriptor Table Register). The GDTR is a 48-bit register that holds the base address of the GDT and its limit.

In protected memory addressing, segment registers hold selectors pointing to entries in the Global Descriptor Table (GDT) or Local Descriptor Table (LDT), specifying memory segments for data, code, and stack.

6. Explain the instruction w.r.t 8086 microprocessor MOV AX, [BX]

The instruction MOV AX, [BX] in the context of the 8086 microprocessor is an example of a memory-to-register transfer. It signifies that the content of the memory location addressed by the BX register is moved into the AX register.

Here's a breakdown:

- **MOV:** This mnemonic stands for "move" and is used for data transfer operations.
- **AX:** It is one of the general-purpose registers in the 8086 microprocessor, specifically the 16-bit accumulator register.
- **[BX]:** Square brackets indicate that the value inside the brackets is treated as a memory address. In this case, it refers to the memory location addressed by the contents of the BX register.

1.

I. How can you determine the microprocessor as 8-bit or 16-bit or 32-bit or 64-bit?

To determine the microprocessor's bit architecture, check its data bus width. 8-bit has an 8-bit data bus, 16-bit has 16 bits, 32-bit has 32 bits, and 64-bit has 64 bits.

II. Distinguish between 8085 and 8086.

Already noted.

III. Mention the role of segment register during protected mode operation.

In protected mode, segment registers define memory segments. They play a crucial role in providing access control and memory protection by specifying the base address of the segments.

IV. Distinguish microcontroller and microprocessor.

Microprocessor	Microcontroller
1. Contains only CPU; RAM, ROM, I/O, timer are separately interfaced.	1. CPU, RAM, ROM, I/O and timer are all on a single chip
2. Designer decides on the amount of ROM, RAM and I/O ports.	2. Fix amount of on-chip ROM, RAM, I/O ports
3. High cost	3. Low cost
4. General-purpose	4. Single-purpose
5. High speed	5. Low speed
6. Higher Power Consumption	6. Low Power Consumption
ex – 8085, 8086	ex – 8051, AVR

V. Shortly describe 82C55 PPI with its operational modes.

82C55 PPI is a programmable peripheral interface. It has three operational modes: Mode 0 (basic I/O), Mode 1 (strobed I/O), and Mode 2 (bi-directional I/O).

VI. Why is memory decoding necessary in computer system?

Memory decoding ensures proper addressing of memory locations, preventing conflicts. It's necessary for the CPU to access specific memory locations in a computer system.

Session:2017-2018

1. What's wrong with the MOV DS, SS Instruction? What do the following MOV instructions accomplish?

The instruction MOV DS, SS is not a valid x86 assembly instruction. The MOV (move) instruction is used to copy data from one location to another, but there is no direct instruction to move data between the DS (Data Segment) and SS (Stack Segment) registers. The DS and SS registers typically point to different segments in memory, and their values are usually set during program initialization.

- I. **MOV AX,BX:** This instruction copies the contents of the BX register into the AX register. It is a basic move operation between two general-purpose registers.
- II. **MOV BX,AX:** This instruction copies the contents of the AX register into the BX register. It is another basic move operation between two general-purpose registers.
- III. **MOV ESP,EBP:** This instruction copies the contents of the EBP (Base Pointer) register into the ESP (Stack Pointer) register. This operation is often seen in function prologues and epilogues, where the base pointer is typically used to reference parameters and local variables on the stack.
- IV. **MOV RAX,RCX:** This instruction copies the contents of the RCX register into the RAX register. In 64-bit x86 architecture, RAX and RCX are 64-bit general-purpose registers, and this operation is a move between two 64-bit registers.

2. What is the difference between an intersegment and intra-segment jump? Show which JMP instruction assembles (short, near or far) if the JMP THERE instruction is stored at memory address 10000H and the address of THERE is:

- III. 10020H
- IV. 11000H
- V. 0FFFFEH
- VI. 30000H

Aspect	Intersegment Jump	Intra-segment Jump
Definition	Jump between different code segments.	Jump within the same code segment.
Addressing	Involves a new segment address.	Uses addresses within the current segment.
Memory Access	Can access different memory segments.	Limited to the memory allocated within the current segment.
Usage	Used for accessing code/data in different memory areas.	Typically used for loops, conditionals within the same code.
Example	JMP 2000:0100 (Jumps to segment 2000, offset 0100).	JMP 0100 (Jumps to offset 0100 within the current segment).

- **(a) 10020H**
10000H
00020H → 0000 0000 0000 0010 0000
1 byte
1 byte displacement → short jump
- **(b) 11000H**
10000H
01000H → 0000 0001 0000 0000 0000
2 byte
2 byte displacement → near jump
- **(c) 0FFFEH**
10000H
2H → 0000 0010
1 byte
1 byte displacement → short jump
- **(d) 30000H**
10000H
20000H → 0000 0010 0000 0000 0000 0000
3 byte
3 byte displacement → far jump

3. Convert an 8B9E004CH from machine language to assembly language. IF a MOV SL[BX+2] instruction appears in a program, what is its machine language equivalent?

- The machine language 8B9E004C can be represented in x86 assembly as:
 - MOV BX, [SI+4C00]
- The machine language equivalent for MOV SL[BX+2] in x86 assembly is: 8B 47 02